

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Мобильная разработка

Отчет
Курсовая работа

Выполнил:

Чебан Илья

Группа

К3441

Проверил:

Шатинский Г.С.

Санкт-Петербург
2026 г.

Отчёт по курсовой работе

1. Тема

Разработка Android-мессенджера с архитектурой MVVM

Стек: Kotlin, Android SDK, AndroidX, Navigation Component, ViewModel и LiveData, Room, Retrofit, Material Components for Android.

Задача 1 - навигация по вкладкам

В рамках курсовой работы требовалось создать простое Android-приложение «Мессенджер», содержащее одну главную Activity и навигацию, основанную на использовании Fragment. Переходы между экранами должны быть реализованы с помощью Bottom Navigation и Navigation Component. Приложение обязано включать экраны новостей, профиля и настроек. Дополнительно необходимо организовать логирование жизненного цикла Activity и Fragment.

Реализация

В качестве точки входа в приложение используется MainActivity, в которой размещена панель нижней навигации. Для управления переходами между экранами был настроен Navigation Graph, обеспечивающий навигацию между тремя фрагментами:

- FeedFragment - экран ленты сообщений (реализован в виде заглушки);
- ProfileFragment - экран профиля пользователя;
- SettingsFragment - экран настроек приложения с возможностью переключения темы.

Управление навигацией осуществляется через NavController из Navigation Component. Для отслеживания корректной работы приложения реализовано логирование основных этапов жизненного цикла

Задача 2 - MVVM и управление состоянием профиля и настроек

Требовалось реализовать архитектуру MVVM для управления состоянием приложения. Данные профиля пользователя и параметры оформления должны храниться во ViewModel и корректно сохраняться при изменении конфигурации, например при

повороте экрана. Экран профиля должен позволять редактировать имя и статус пользователя, а экран настроек - изменять тему приложения. Обновление пользовательского интерфейса должно выполняться реактивно с помощью LiveData. Также необходимо реализовать логирование жизненного цикла ViewModel.

Реализация

1. Использование ViewModel

Для экранов профиля и настроек были созданы отдельные ViewModel, в которых хранятся пользовательские данные и выбранные параметры приложения. Это позволило отделить бизнес-логику от пользовательского интерфейса и упростить управление состоянием.

2. Экран профиля

На экране профиля отображаются имя и статус пользователя, доступные для редактирования через поля ввода. Все изменения сохраняются во ViewModel. Благодаря использованию LiveData интерфейс автоматически обновляется при изменении данных, что обеспечивает корректную работу экрана при пересоздании фрагмента.

3. Экран настроек и переключение темы

В настройках реализован переключатель между светлой и тёмной темами. Выбранное значение сохраняется во ViewModel и применяется ко всему приложению с использованием AppCompatActivity, что демонстрирует практическое применение MVVM для работы с глобальными настройками.

4. Сохранение состояния

Хранение данных во ViewModel обеспечивает сохранение состояния приложения при изменении конфигурации. Пользовательские данные и выбранные настройки не теряются, что повышает удобство использования.

5. Логирование жизненного цикла ViewModel

В ViewModel добавлено логирование моментов создания и уничтожения объектов, что позволяет контролировать корректность работы архитектуры и отслеживать освобождение ресурсов.

Задача 3 - Загрузка сообщений из API и Room для офлайн доступа

Необходимо реализовать загрузку сообщений из внешнего API с последующим сохранением данных в локальной базе Room для обеспечения работы приложения без

подключения к интернету. Сетевое взаимодействие должно быть реализовано с помощью Retrofit и корутин. Отображение сообщений требуется выполнить на экране «Лента» с использованием RecyclerView. Репозиторий должен выступать единой точкой доступа ViewModel к данным.

Реализация

1. Репозиторий сообщений

В проекте реализован репозиторий, который объединяет работу с удалённым API и локальной базой данных Room. ViewModel получает данные исключительно через репозиторий, что соответствует принципу единого источника истины.

2. Работа с сетью

Для выполнения сетевых запросов используется Retrofit с поддержкой корутин. Это обеспечивает асинхронную загрузку данных без блокировки главного потока и упрощает обработку ошибок.

3. Отображение ленты

Сообщения отображаются на экране «Лента» с помощью RecyclerView. Каждый элемент списка содержит основную информацию о сообщении и удобен для просмотра пользователем.

4. Локальное хранение данных

После успешной загрузки данные сохраняются в базу Room, что позволяет повторно использовать информацию без повторных сетевых запросов.

5. Поддержка офлайн-режима

При отсутствии интернет-соединения данные автоматически загружаются из локальной базы, обеспечивая стабильную работу приложения в офлайн-режиме.

6. Обновление данных вручную

На экране ленты реализована кнопка обновления, запускающая повторную загрузку сообщений через ViewModel. При этом данные обновляются как в интерфейсе, так и в локальной базе.

Задача 4 - Улучшение интерфейса и фоновой синхронизации

Требовалось улучшить пользовательский интерфейс приложения с использованием компонентов Material Design и расширить функциональность. Необходимо реализовать интерактивный список сообщений с отображением аватаров, имени автора и возможностью ставить лайки. Также требовалось настроить фоновую синхронизацию данных с помощью WorkManager и реализовать отправку уведомлений при успешном обновлении. Приложение должно корректно работать в фоновом режиме, запрашивать необходимые разрешения и учитывать изменение состояния сети, сохраняя архитектуру MVVM.

Реализация

Интерфейс списка сообщений:

- цветные аватары с первой буквой имени пользователя;
- отображение имени автора и текста сообщения;
- лайки и дизлайки с изменяемыми счётчиками;
- отображение количества просмотров.

Использование Material Design:

- MaterialCardView для элементов списка;
- FloatingActionButton для обновления данных;
- AppBarLayout и Toolbar для оформления верхней панели.

Механизм лайков и дизлайков:

- нажатие на иконку изменяет её состояние;
- счётчики увеличиваются и уменьшаются в зависимости от действия;
- лайк и дизлайк являются взаимоисключающими.

Фоновая синхронизация:

- реализована с помощью WorkManager;
- выполняется каждые 15 минут;
- запускается только при наличии интернет-соединения;
- пользовательские реакции (лайки и дизлайки) сохраняются при обновлении данных.

Уведомления:

- создан NotificationChannel для Android 8 и выше;
- после успешной синхронизации отображается уведомление «Новые данные получены»;
- уведомления работают даже в фоновом режиме.

Запрос разрешений:

При запуске приложения запрашивается разрешение на доступ к контактам и отправку уведомлений.

5 Листинг кода

[AppDatabase.kt](#)

```
package com.example.lab1cheban.data

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(entities = [Message::class], version = 1, exportSchema =
false)
abstract class AppDatabase : RoomDatabase() {
    abstract fun messageDao(): MessageDao

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "main_db"
                )
                    .fallbackToDestructiveMigration()
                    .build()
                INSTANCE = instance
                instance
            }
        }
    }
}
```

[Message.kt](#)

```
package com.example.lab1cheban.data

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "messages")
data class Message(
    @PrimaryKey val id: Int,
    val name: String,
    val email: String,
    val body: String
)
```

[MessageApiService.kt](#)

```
package com.example.lab1cheban.data

import retrofit2.http.GET
import retrofit2.http.Query

interface MessageApiService {
    @GET("comments")
    suspend fun getMessages(
        @Query("_page") page: Int,
        @Query("_limit") limit: Int
    ): List<Message>
}
```

[MessageDao.kt](#)

```
package com.example.lab1cheban.data

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query

@Dao
interface MessageDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertMessages(messages: List<Message>)

    @Query("SELECT * FROM messages ORDER BY id DESC")
    suspend fun getAllMessages(): List<Message>

    @Query("DELETE FROM messages")
    suspend fun clearMessages()
}
```

[MessageRepository.kt](#)

```
package com.example.lab1cheban.data

class MessageRepository(private val api: MessageApiService,
    private val db: AppDatabase) {
    private val dao = db.messageDao()

    suspend fun fetchMessages(page: Int, limit: Int): List<Message>
    {
        return try {
            val remote = api.getMessages(page, limit)
        }
```

```

        dao.clearMessages()
        dao.insertMessages(remote)
        remote
    } catch (e: Exception) {
        dao.getAllMessages()
    }
}
}

```

[RetrofitInstance.kt](#)

```

package com.example.lab1cheban.data

import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

object RetrofitInstance {
    private const val BASE_URL =
        "https://jsonplaceholder.typicode.com/"

    val api: MessageApiService by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(MessageApiService::class.java)
    }
}

```

[FeedFragment.kt](#)

```

package com.example.lab1cheban.ui.feed

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import androidx.fragment.app.Fragment
import androidx.lifecycle.LifecycleScope
import androidx.lifecycle.ViewModelProvider
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
import com.example.messenger.R
import
com.google.android.material.floatingactionbutton.FloatingActionBut
ton
import com.google.android.material.snackbar.Snackbar

```



```

import kotlinx.coroutines.flow.collectLatest
import kotlinx.coroutines.launch

class FeedFragment : Fragment() {
    private lateinit var viewModel: FeedViewModel
    private lateinit var adapter: MessageAdapter
    private var currentSnackbar: Snackbar? = null

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        return inflater.inflate(R.layout.fragment_feed, container,
false)
    }

    override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        viewModel = ViewModelProvider(
            this,
            ViewModelProvider.AndroidViewModelFactory.getInstance(requireActiv
ity().application)
        ).get(FeedViewModel::class.java)

        adapter = MessageAdapter(emptyList()) { messageId ->
            viewModel.toggleLike(messageId)
        }

        val recyclerView =
view.findViewById<RecyclerView>(R.id.message_list)
        recyclerView.layoutManager =
LinearLayoutManager(requireContext())
        recyclerView.adapter = adapter

        val fabRefresh =
view.findViewById<FloatingActionButton>(R.id.fab_refresh)
        val btnNext = view.findViewById<Button>(R.id.next_button)
        val btnBack = view.findViewById<Button>(R.id.back_button)

        fabRefresh.setOnClickListener {
            viewModel.refresh()
        }

        btnNext.setOnClickListener { viewModel.nextPage() }
        btnBack.setOnClickListener { viewModel.prevPage() }
    }
}

```

```

        viewLifecycleOwner.lifecycleScope.launch {
            viewModel.messages.collectLatest {
                adapter.updateData(it)
            }
        }

        viewLifecycleOwner.lifecycleScope.launch {
            viewModel.isLoading.collectLatest { isLoading ->
                if (isLoading) {
                    currentSnackbar?.dismiss()
                    currentSnackbar = Snackbar.make(
                        view,
                        "Происходит обновление...",
                        Snackbar.LENGTH_INDEFINITE
                    )
                    currentSnackbar?.show()
                } else {
                    currentSnackbar?.dismiss()
                    currentSnackbar = null
                }
            }
        }

        viewLifecycleOwner.lifecycleScope.launch {
            var previousLoading = false
            viewModel.isLoading.collectLatest { isLoading ->
                if (previousLoading && !isLoading &&
                    viewModel.messages.value.isNotEmpty()) {
                    Snackbar.make(
                        view,
                        "Данные обновлены",
                        Snackbar.LENGTH_SHORT
                    ).show()
                }
                previousLoading = isLoading
            }
        }

        viewModel.loadPage(1)
    }

    override fun onDestroyView() {
        super.onDestroyView()
        currentSnackbar?.dismiss()
    }
}

```

[FeedViewModel.kt](#)

```
package com.example.lab1cheban.ui.feed

import android.app.Application
import androidx.lifecycle.AndroidViewModel
import androidx.lifecycle.viewModelScope
import androidx.work.*
import com.example.lab1cheban.data.AppDatabase
import com.example.lab1cheban.data.Message
import com.example.lab1cheban.data.MessageRepository
import com.example.lab1cheban.data.RetrofitInstance
import com.example.lab1cheban.worker.MessageSyncWorker
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.launch
import java.util.concurrent.TimeUnit

class FeedViewModel(application: Application) :
    AndroidViewModel(application) {
    private val repository = MessageRepository(
        RetrofitInstance.api,
        AppDatabase.getDatabase(application)
    )

    private val _messages =
        MutableStateFlow<List<Message>>(emptyList())
    val messages: StateFlow<List<Message>> get() = _messages

    private val _isLoading = MutableStateFlow(false)
    val isLoading: StateFlow<Boolean> get() = _isLoading

    private var currentPage = 1
    private val limit = 5

    init {
        setupPeriodicSync()
    }

    private fun setupPeriodicSync() {
        val constraints = Constraints.Builder()
            .setRequiredNetworkType(NetworkType.CONNECTED)
            .build()

        val syncRequest =
            PeriodicWorkRequestBuilder<MessageSyncWorker>(
                15, TimeUnit.MINUTES
            )
            .setConstraints(constraints)
```

```

        .setBackoffCriteria(
            BackoffPolicy.LINEAR,
            WorkRequest.MIN_BACKOFF_MILLIS,
            TimeUnit.MILLISECONDS
        )
        .build()

WorkManager.getInstance(getApplication()).enqueueUniquePeriodicWork(
    "message_sync",
    ExistingPeriodicWorkPolicy.KEEP,
    syncRequest
)
}

fun loadPage(page: Int, showNotification: Boolean = false) {
    viewModelScope.launch {
        if (showNotification) {
            _isLoading.value = true
        }
        val data = repository.fetchMessages(page, limit)
        _messages.value = data
        currentPage = page
        if (showNotification) {
            _isLoading.value = false
        }
    }
}

fun nextPage() {
    loadPage(currentPage + 1)
}

fun prevPage() {
    if (currentPage > 1) {
        loadPage(currentPage - 1)
    }
}

fun refresh() {
    loadPage(1, showNotification = true)
}

fun toggleLike(messageId: Int) {
}

override fun onCleared() {

```

```
        super.onCleared()
    }
}
```

[MessageAdapter.kt](#)

```
package com.example.lab1cheban.ui.feed

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.ImageView
import android.widget.TextView
import androidx.recyclerview.widget.RecyclerView
import com.example.messenger.R
import com.example.lab1cheban.data.Message

class MessageAdapter(
    private var items: List<Message>,
    private val onLikeClick: (Int) -> Unit
) : RecyclerView.Adapter<MessageAdapter.MessageViewHolder>() {

    private val likedMessages = mutableSetOf<Int>()

    class MessageViewHolder(view: View) :
        RecyclerView.ViewHolder(view) {
        val avatar: ImageView = view.findViewById(R.id.item_avatar)
        val name: TextView = view.findViewById(R.id.item_user)
        val email: TextView = view.findViewById(R.id.item_content)
        val body: TextView = view.findViewById(R.id.item_timestamp)
        val likeIcon: ImageView = view.findViewById(R.id.item_like)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType:
    Int): MessageViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_message, parent, false)
        return MessageViewHolder(view)
    }

    override fun onBindViewHolder(holder: MessageViewHolder,
    position: Int) {
        val item = items[position]

        holder.avatar.setImageResource(R.drawable.ic_avatar_placeholder)
        holder.name.text = item.name
        holder.email.text = item.email
    }
}
```

```

        holder.body.text = item.body

        val isLiked = likedMessages.contains(item.id)
        holder.likeIcon.setImageResource(
            if (isLiked) R.drawable.ic_favorite_filled
            else R.drawable.ic_favorite_border
        )

        holder.likeIcon.setOnClickListener {
            if (likedMessages.contains(item.id)) {
                likedMessages.remove(item.id)
            } else {
                likedMessages.add(item.id)
            }
            notifyItemChanged(position)
            onLikeClick(item.id)
        }
    }

    override fun getItemCount(): Int = items.size

    fun updateData(newItems: List<Message>) {
        items = newItems
        notifyDataSetChanged()
    }
}

```

[ProfileFragment.kt](#)

```

package com.example.messenger.ui.profile

import android.os.Bundle
import android.util.Log
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import androidx.fragment.app.Fragment
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.lifecycleScope
import com.example.messenger.R
import kotlinx.coroutines.flow.collectLatest
import kotlinx.coroutines.launch

class ProfileFragment : Fragment() {

    private val TAG = "ProfileFragment"
}

```

```

private lateinit var viewModel: ProfileViewModel

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    Log.d(TAG, "onCreate: Fragment создан")
}

override fun onCreateView(
    inflater: LayoutInflater,
    container: ViewGroup?,
    savedInstanceState: Bundle?
): View? {
    Log.d(TAG, "onCreateView: View создан")
    return inflater.inflate(R.layout.fragment_profile,
container, false)
}

override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
    super.onViewCreated(view, savedInstanceState)
    Log.d(TAG, "onViewCreated: View готов к использованию")

    viewModel =
ViewModelProvider(this).get(ProfileViewModel::class.java)

    val nameEditText =
view.findViewById<EditText>(R.id.profile_name_edit)
    val statusEditText =
view.findViewById<EditText>(R.id.profile_status_edit)
    val nameTextView =
view.findViewById<TextView>(R.id.profile_name)
    val statusTextView =
view.findViewById<TextView>(R.id.profile_status)
    val emailTextView =
view.findViewById<TextView>(R.id.profile_email)
    val phoneTextView =
view.findViewById<TextView>(R.id.profile_phone)
    val saveButton = view.findViewById<Button>(R.id.save_button)

    viewLifecycleOwner.lifecycleScope.launch {
        viewModel.profileData.collectLatest { profile ->
            nameTextView.text = profile.name
            statusTextView.text = profile.status
            emailTextView.text = profile.email
            phoneTextView.text = profile.phone
        }
    }
}

```

```
        saveButton.setOnClickListener {
            val newName = nameEditText.text.toString().trim()
            val newStatus = statusEditText.text.toString().trim()

            if (newName.isNotEmpty()) {
                viewModel.updateName(newName)
            }
            if (newStatus.isNotEmpty()) {
                viewModel.updateStatus(newStatus)
            }

            nameEditText.text.clear()
            statusEditText.text.clear()
        }
    }

    override fun onStart() {
        super.onStart()
        Log.d(TAG, "onStart: Fragment запущен")
    }

    override fun onResume() {
        super.onResume()
        Log.d(TAG, "onResume: Fragment возобновлен")
    }

    override fun onPause() {
        super.onPause()
        Log.d(TAG, "onPause: Fragment приостановлен")
    }

    override fun onStop() {
        super.onStop()
        Log.d(TAG, "onStop: Fragment остановлен")
    }

    override fun onDestroyView() {
        super.onDestroyView()
        Log.d(TAG, "onDestroyView: View уничтожен")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "onDestroy: Fragment уничтожен")
    }
}
```


[ProfileViewModel.kt](#)

```
package com.example.messenger.ui.profile

import androidx.lifecycle.ViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow

data class ProfileData(
    val name: String = "Илья Чебан",
    val status: String = "В сети",
    val email: String = "ilyacheban2004@mail.ru",
    val phone: String = "+7-911-123-33-30"
)

class ProfileViewModel : ViewModel() {
    private val _profileData = MutableStateFlow(ProfileData())
    val profileData: StateFlow<ProfileData> =
        _profileData.asStateFlow()

    fun updateName(newName: String) {
        _profileData.value = _profileData.value.copy(name =
newName)
    }

    fun updateStatus(newStatus: String) {
        _profileData.value = _profileData.value.copy(status =
newStatus)
    }
}
```

[SettingsFragment.kt](#)

```
package com.example.messenger.ui.settings

import android.content.Context
import android.os.Bundle
import android.util.Log
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import androidx.fragment.app.Fragment
import com.example.messenger.R
import com.google.android.material.slider.Slider
```

```
import com.google.android.material.snackbar.Snackbar
import com.google.android.material.switchmaterial.SwitchMaterial

class SettingsFragment : Fragment() {

    private val TAG = "SettingsFragment"
    private val PREFS_NAME = "settings_prefs"
    private val PREF_NOTIFICATIONS = "notifications_enabled"
    private val PREF_AUTO_SYNC = "auto_sync_enabled"
    private val PREF_FONT_SIZE = "font_size"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, "onCreate: Fragment создан")
    }

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        Log.d(TAG, "onCreateView: View создан")
        return inflater.inflate(R.layout.fragment_settings,
container, false)
    }

    override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        Log.d(TAG, "onViewCreated: View готов к использованию")

        val prefs =
requireContext().getSharedPreferences(PREFS_NAME,
Context.MODE_PRIVATE)

        val themeSwitch =
view.findViewById<SwitchMaterial>(R.id.theme_switch)
        val notificationsSwitch =
view.findViewById<SwitchMaterial>(R.id.notifications_switch)
        val autoSyncSwitch =
view.findViewById<SwitchMaterial>(R.id.auto_sync_switch)
        val fontSizeSlider =
view.findViewById<Slider>(R.id.font_size_slider)
        val fontSizeLabel =
view.findViewById<TextView>(R.id.font_size_label)
        val clearCacheButton =
view.findViewById<Button>(R.id.clear_cache_button)
```

```

        notificationsSwitch.isChecked =
prefs.getBoolean(PREF_NOTIFICATIONS, true)
        autoSyncSwitch.isChecked = prefs.getBoolean(PREF_AUTO_SYNC,
true)
        fontSizeSlider.value = prefs.getInt(PREF_FONT_SIZE,
1).toFloat()

        updateFontSizeLabel(fontSizeLabel,
fontSizeSlider.value.toInt())

        themeSwitch.setOnCheckedChangeListener { _, isChecked ->
            if (isChecked) {
AppCompatActivity.setDefaultNightMode(AppCompatActivity.MODE_NIGHT_
YES)
                Log.d(TAG, "Темная тема включена")
            } else {
AppCompatActivity.setDefaultNightMode(AppCompatActivity.MODE_NIGHT_
NO)
                Log.d(TAG, "Светлая тема включена")
            }
        }

        notificationsSwitch.setOnCheckedChangeListener { _,
isChecked ->
            prefs.edit().putBoolean(PREF_NOTIFICATIONS,
isChecked).apply()
            Log.d(TAG, "Уведомления: ${if (isChecked) "включены"
else "выключены"}")
            Snackbar.make(
                view,
                if (isChecked) "Уведомления включены" else
"Уведомления выключены",
                Snackbar.LENGTH_SHORT
            ).show()
        }

        autoSyncSwitch.setOnCheckedChangeListener { _, isChecked ->
            prefs.edit().putBoolean(PREF_AUTO_SYNC,
isChecked).apply()
            Log.d(TAG, "Автообновление: ${if (isChecked) "включено"
else "выключено"}")
            Snackbar.make(
                view,
                if (isChecked) "Автообновление включено" else
"Автообновление выключено",
                Snackbar.LENGTH_SHORT

```

```

        ).show()
    }

    fontSizeSlider.addOnChangeListener { _, value, _ ->
        val size = value.toInt()
        prefs.edit().putInt(PREF_FONT_SIZE, size).apply()
        updateFontSizeLabel(fontSizeLabel, size)
        Log.d(TAG, "Размер шрифта изменен: $size")
    }

    clearCacheButton.setOnClickListener {
        requireContext().cacheDir.deleteRecursively()
        Snackbar.make(view, "Кэш очищен",
Snackbar.LENGTH_SHORT).show()
        Log.d(TAG, "Кэш очищен")
    }
}

private fun updateFontSizeLabel(label: TextView, size: Int) {
    label.text = when (size) {
        0 -> "Маленький"
        1 -> "Средний"
        2 -> "Большой"
        else -> "Средний"
    }
}

override fun onStart() {
    super.onStart()
    Log.d(TAG, "onStart: Fragment запущен")
}

override fun onResume() {
    super.onResume()
    Log.d(TAG, "onResume: Fragment возобновлен")
}

override fun onPause() {
    super.onPause()
    Log.d(TAG, "onPause: Fragment приостановлен")
}

override fun onStop() {
    super.onStop()
    Log.d(TAG, "onStop: Fragment остановлен")
}

override fun onDestroyView() {

```

```

        super.onDestroyView()
        Log.d(TAG, "onDestroyView: View уничтожен")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "onDestroy: Fragment уничтожен")
    }
}

```

[MessageSyncWorker.kt](#)

```

package com.example.lab1cheban.worker

import android.app.NotificationChannel
import android.app.NotificationManager
import android.content.Context
import android.os.Build
import androidx.core.app.NotificationCompat
import androidx.work.CoroutineWorker
import androidx.work.WorkerParameters
import com.example.lab1cheban.data.AppDatabase
import com.example.lab1cheban.data.MessageRepository
import com.example.lab1cheban.data.RetrofitInstance
import com.example.messenger.R

class MessageSyncWorker(
    context: Context,
    params: WorkerParameters
) : CoroutineWorker(context, params) {

    companion object {
        private const val CHANNEL_ID = "message_sync_channel"
        private const val NOTIFICATION_ID = 1001
    }

    override suspend fun doWork(): Result {
        return try {
            val repository = MessageRepository(
                RetrofitInstance.api,
                AppDatabase.getDatabase(applicationContext)
            )

            val messages = repository.fetchMessages(page = 1, limit
= 10)

            if (messages.isNotEmpty()) {
                showNotification("Новые данные получены",

```

```

        "Синхронизировано ${messages.size} сообщений")
    }

    Result.success()
} catch (e: Exception) {
    e.printStackTrace()
    Result.retry()
}
}

private fun showNotification(title: String, message: String) {
    val notificationManager = applicationContext
        .getSystemService(Context.NOTIFICATION_SERVICE) as
NotificationManager

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val channel = NotificationChannel(
            CHANNEL_ID,
            "Синхронизация сообщений",
            NotificationManager.IMPORTANCE_DEFAULT
        ).apply {
            description = "Уведомления о синхронизации
сообщений"
        }
        notificationManager.createNotificationChannel(channel)
    }

    val notification =
NotificationCompat.Builder(applicationContext, CHANNEL_ID)
        .setSmallIcon(R.drawable.ic_notification)
        .setContentTitle(title)
        .setContentText(message)
        .setPriority(NotificationCompat.PRIORITY_DEFAULT)
        .setAutoCancel(true)
        .build()

    notificationManager.notify(NOTIFICATION_ID, notification)
}
}

```

6 xml ЛИСТИНГ

item_message.xml

```

<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView

```

```
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:layout_margin="8dp"
app:cardElevation="4dp"
app:cardCornerRadius="12dp">
```

```
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:padding="16dp">
```

```
<ImageView
    android:id="@+id/item_avatar"
    android:layout_width="48dp"
    android:layout_height="48dp"
    android:contentDescription="Аватар пользователя"
    android:src="@drawable/ic_avatar_placeholder"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
```

```
<TextView
    android:id="@+id/item_user"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginStart="12dp"
    android:layout_marginEnd="8dp"
    android:text="User"
    android:textStyle="bold"
    android:textSize="16sp"
    android:textColor="?attr/colorOnSurface"
    app:layout_constraintStart_toEndOf="@id/item_avatar"
    app:layout_constraintTop_toTopOf="@id/item_avatar"
    app:layout_constraintEnd_toStartOf="@id/item_like" />
```

```
<TextView
    android:id="@+id/item_content"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginStart="12dp"
    android:text="Email"
    android:textSize="12sp"
    android:textColor="?attr/colorOnSurfaceVariant"
    app:layout_constraintTop_toBottomOf="@id/item_user"
    app:layout_constraintStart_toEndOf="@id/item_avatar"
    app:layout_constraintEnd_toEndOf="parent"/>
```

```

<TextView
    android:id="@+id/item_timestamp"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:text="Message body"
    android:textSize="14sp"
    android:textColor="?attr/colorOnSurface"
    app:layout_constraintTop_toBottomOf="@id/item_avatar"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />

<ImageView
    android:id="@+id/item_like"
    android:layout_width="24dp"
    android:layout_height="24dp"
    android:contentDescription="Лайк"
    android:src="@drawable/ic_favorite_border"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>

</com.google.android.material.card.MaterialCardView>

```

fragment_settings.xml

```

<?xml version="1.0" encoding="utf-8"?>
<ScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="16dp">

        <TextView
            android:id="@+id/settings_title"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Настройки"
            android:textSize="24sp"
            android:textStyle="bold"
            app:layout_constraintEnd_toEndOf="parent"

```



```

        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

<LinearLayout
    android:id="@+id/settings_container"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="32dp"
    android:orientation="vertical"

app:layout_constraintTop_toBottomOf="@id/settings_title">

    <com.google.android.material.card.MaterialCardView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="8dp"
        app:cardElevation="2dp"
        app:cardCornerRadius="8dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:padding="16dp">

            <TextView
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:text="Темная тема"
                android:textSize="18sp" />

<com.google.android.material.switchmaterial.SwitchMaterial
    android:id="@+id/theme_switch"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content" />

    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="8dp"
    app:cardElevation="2dp"
    app:cardCornerRadius="8dp">

```

```
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:padding="16dp">

            <TextView
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:text="Уведомления"
                android:textSize="18sp" />

<com.google.android.material.switchmaterial.SwitchMaterial
    android:id="@+id/notifications_switch"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:checked="true" />

    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="8dp"
    app:cardElevation="2dp"
    app:cardCornerRadius="8dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:padding="16dp">

        <TextView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Автообновление"
            android:textSize="18sp" />

<com.google.android.material.switchmaterial.SwitchMaterial
    android:id="@+id/auto_sync_switch"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
```

```
        android:checked="true" />

    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="24dp"
    android:layout_marginBottom="8dp"
    android:text="Размер шрифта"
    android:textSize="16sp"
    android:textStyle="bold" />

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="8dp"
    app:cardElevation="2dp"
    app:cardCornerRadius="8dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <TextView
            android:id="@+id/font_size_label"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Средний"
            android:textSize="16sp"
            android:layout_gravity="center" />

        <com.google.android.material.slider.Slider
            android:id="@+id/font_size_slider"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:valueFrom="0"
            android:valueTo="2"
            android:stepSize="1"
            android:value="1" />

    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<TextView
```

```

        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="24dp"
        android:layout_marginBottom="8dp"
        android:text="0 приложений"
        android:textSize="16sp"
        android:textStyle="bold" />

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="8dp"
    app:cardElevation="2dp"
    app:cardCornerRadius="8dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Версия: 1.0.0"
            android:textSize="14sp" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:text="Разработчик: Илья Чебан"
            android:textSize="14sp" />

    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<Button
    android:id="@+id/clear_cache_button"

style="@style/Widget.Material3.Button.OutlinedButton"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:layout_marginBottom="16dp"
    android:text="Очистить кэш" />

</LinearLayout>

```

```
        </androidx.constraintlayout.widget.ConstraintLayout>
    </ScrollView>
```

fragment_profile.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">

    <TextView
        android:id="@+id/profile_title"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Профиль"
        android:textSize="24sp"
        android:textStyle="bold"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <ImageView
        android:id="@+id/profile_avatar"
        android:layout_width="100dp"
        android:layout_height="100dp"
        android:layout_marginTop="32dp"
        android:background="@android:color/darker_gray"
        android:contentDescription="Аватар пользователя"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/profile_title" />

    <TextView
        android:id="@+id/profile_name"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="Илья Чебан"
        android:textSize="20sp"
        android:textStyle="bold"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/profile_avatar" />
```

```
<TextView
    android:id="@+id/profile_status"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="4dp"
    android:text="В сети"
    android:textSize="14sp"
    android:textColor="@android:color/darker_gray"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/profile_name" />

<TextView
    android:id="@+id/profile_email"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:text="ilyacheban2004@mail.ru"
    android:textSize="16sp"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/profile_status" />

<TextView
    android:id="@+id/profile_phone"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:text="+7-911-123-33-30"
    android:textSize="16sp"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/profile_email" />

<TextView
    android:id="@+id/edit_section_title"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="32dp"
    android:text="Редактировать профиль"
    android:textSize="18sp"
    android:textStyle="bold"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/profile_phone" />

<EditText
    android:id="@+id/profile_name_edit"
```

```

        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:hint="Введите новое имя"
        android:inputType="textPersonName"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"

app:layout_constraintTop_toBottomOf="@id/edit_section_title" />

    <EditText
        android:id="@+id/profile_status_edit"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:hint="Введите новый статус"
        android:inputType="text"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/profile_name_edit"
    />

    <Button
        android:id="@+id/save_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="Сохранить"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"

app:layout_constraintTop_toBottomOf="@id/profile_status_edit" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

7 Фото приложения

Страница настроек

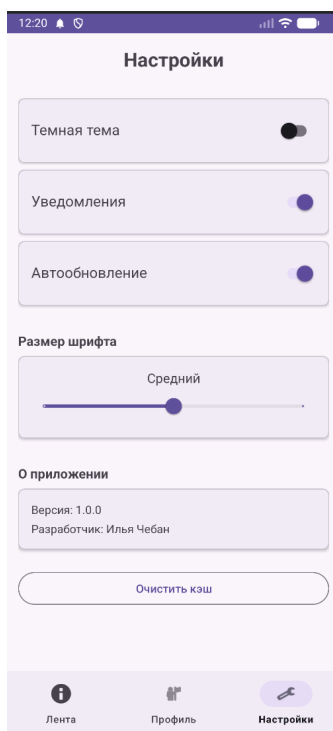


Рисунок 1 - Страница Настроек

Страница профиля

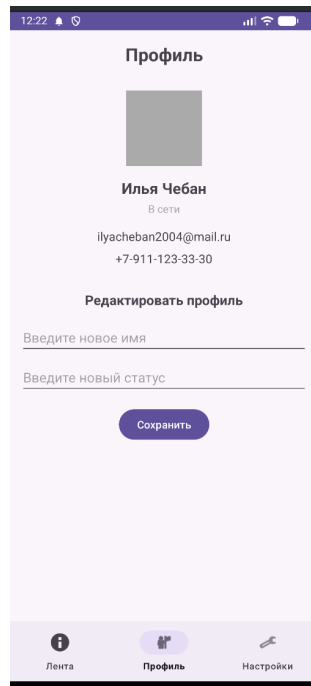


Рисунок 2 - страница Профиля

Страница Ленты

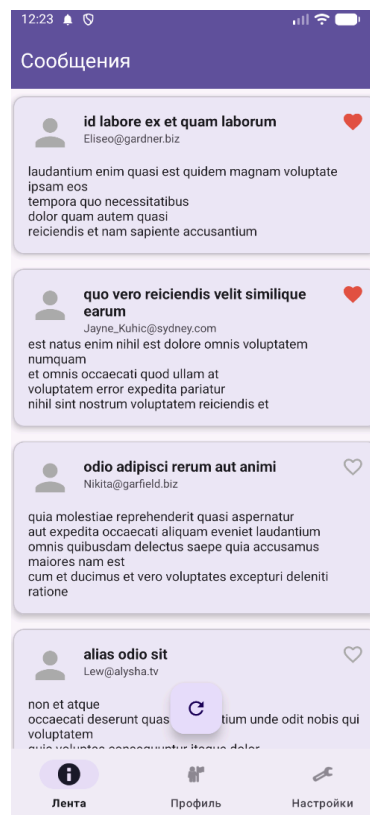


Рисунок 3 - Страница Ленты

8 Вывод по работе

В ходе курсовой работы было создано Android-приложение с использованием архитектуры MVVM и современных инструментов платформы Android. Реализована загрузка данных из сети, их сохранение в локальной базе и отображение в интерфейсе с интерактивными элементами. Разработанное приложение может служить основой для дальнейшего расширения функциональности. Полученные результаты подтверждают освоение ключевых принципов разработки современных Android-приложений